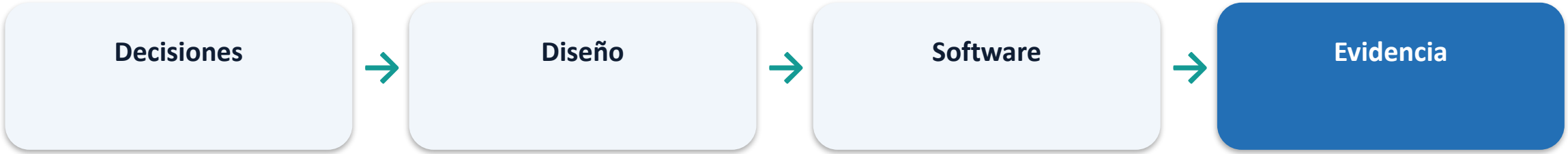
Evaluación 3 | Arquitectura que ya funciona

Diseño actualizado + primer incremento ejecutable



No se espera el producto final. Sí se espera una parte real, coherente y demostrable.

Qué cambia respecto al plan anterior

Ahora: Evaluación 3

Arquitectura revisada, decisiones justificadas y primera iteración funcional.

Después: Evaluación 4

Solicitud de cambio, análisis de impacto, evolución y cierre del producto.

No se evalúa aún

Implementación completa de todas las funcionalidades del RFP.

Sí se evalúa

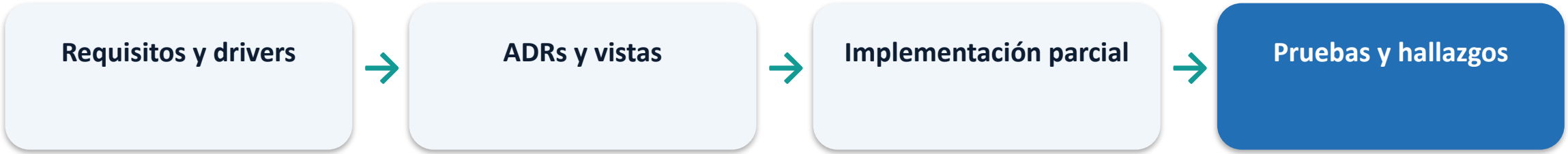
Que el software construido corresponda a lo diseñado y pueda verificarse.

La pregunta central

¿La arquitectura propuesta se convirtió en un incremento de software real y verificable?

La documentación explica las decisiones; la ejecución demuestra su viabilidad.

De la Evaluación 2 a la 3



Actualizar lo que cambió: no entregar diagramas antiguos como si describieran el software actual.

Qué debe estar funcionando

Flujo extremo a extremo

Al menos un caso de uso significativo, con entrada, reglas y resultado verificable.

Persistencia / estado

El sistema conserva o gestiona estado cuando el caso lo necesita.

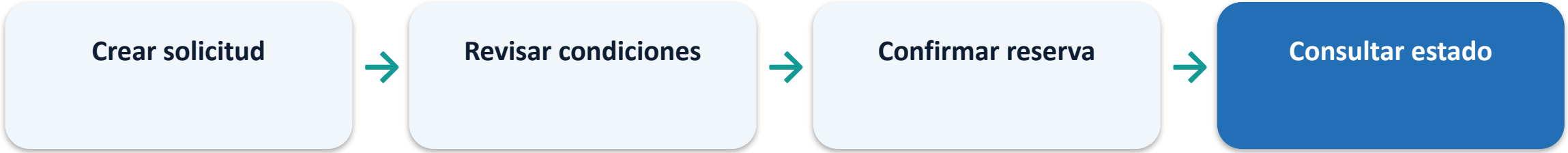
Reglas del dominio

Validaciones, permisos o transiciones relevantes; no solo pantallas.

Ejecución reproducible

Pasos claros para instalar, iniciar, probar y reproducir la demostración.

Ejemplo DH-00: primer incremento



Ejemplo de evidencia: una solicitud válida se confirma; una reserva conflictiva se rechaza.

No hace falta implementar todavía notificaciones reales, reportes completos ni todas las políticas.

Ejemplo: la arquitectura se refleja en el código

Vista estructural

API → servicio de reservas → repositorio → base de datos.

Decisión (ADR)

Persistencia relacional para controlar conflictos y conservar historial.

Implementación

Regla de conflicto ubicada en la capa adecuada; no duplicada en la interfaz.

Validación

Prueba de reserva válida + prueba de solapamiento y resultado esperado.

Documentación de arquitectura actualizada

Vistas

Contexto y contenedores actualizados; componentes y despliegue cuando aporten valor.

Decisiones

ADRs vigentes: aceptadas, modificadas o reemplazadas; consecuencias reales.

Diseño interno

Módulos, responsabilidades, interfaces, modelo de dominio/ datos y flujo crítico.

Trazabilidad

Requisito → decisión → componente → implementación → prueba.

Qué diagramas mostrar

Contexto + contenedores

Quién interactúa y cuáles son los bloques principales del sistema.

Componentes / módulos

Dónde viven las reglas y qué responsabilidades tiene cada módulo.

Secuencia / estados

Cómo funciona el caso de uso implementado y sus errores relevantes.

Datos / despliegue

Entidades, relaciones y entorno de ejecución, cuando sean significativos.

No se premia la cantidad de diagramas: se premia su utilidad y consistencia.

Decisiones: diseño previsto vs diseño real

Decisión inicial

“Usaremos un monolito modular con módulos de solicitudes y reservas”.

Hallazgo al implementar

La confirmación también necesita controlar concurrencia y registrar cambios.

Decisión refinada

Definir frontera transaccional, validación de solapamiento y auditoría.

Evidencia

ADR revisado + diagrama de secuencia + prueba de conflicto.

Demostrar, no solo afirmar

Afirmación débil

“El sistema es seguro y escalable”.

Evidencia concreta

“Un usuario sin permiso recibe 403; se incluye prueba y resultado”.

Afirmación débil

“Las reservas no se duplican”.

Evidencia concreta

“Dos solicitudes concurrentes no confirman el mismo espacio/horario”.

Pruebas y límites del incremento

Pruebas mínimas útiles

Caso exitoso, caso inválido y al menos un riesgo relevante del dominio.

Cómo ejecutarlas

Comandos, datos de prueba, entorno, resultado y fecha/versión.

Qué no funciona todavía

Distinguir implementado, simulado, diseñado y pendiente.

Riesgos abiertos

Registrar defectos, supuestos no validados y próximos pasos.

El ZIP: una entrega autocontenida

Estructura sugerida (puede variar si todo es fácil de localizar):

```
evaluacion-3/
  README.md                ← índice y pasos para ejecutar
  docs/                    ← arquitectura, ADRs, diagramas, trazabilidad
  src/                     ← incremento de software
  tests/                   ← pruebas y evidencia de ejecución
  evidence/                ← capturas, resultados, limitaciones
  video-link-or-video.txt  ← referencia al video o archivo incluido
```

Entregar un único ZIP por equipo; no depender de rutas locales ni archivos externos inaccesibles.

README: permitir que otro reproduzca el trabajo

Identidad y alcance

Equipo, proyecto DH-01…DH-05, versión y caso de uso implementado.

Preparación

Requisitos, dependencias, configuración sintética y carga de datos.

Ejecución

Comandos para iniciar, usuario de prueba, URL/puerto y recorrido.

Verificación

Cómo ejecutar tests, resultados esperados y limitaciones conocidas.

Video obligatorio: 5–6 minutos

0:00–0:45 | Contexto

Problema, alcance del incremento y qué funciona hoy.

0:45–2:00 | Arquitectura

Mostrar vistas y las decisiones principales que se materializaron.

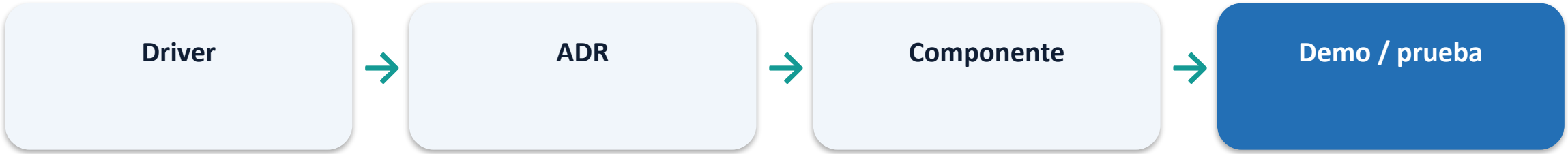
2:00–4:45 | Demostración

Ejecutar el flujo; enseñar validación y un caso alternativo/error.

4:45–6:00 | Evidencia

Pruebas, divergencias entre diseño y código, pendientes y riesgos.

En el video: conectar decisión y resultado



Ejemplo DH-00: evitar reservas simultáneas → control transaccional → módulo Reservas → prueba de conflicto.

Evitar una grabación que sea solo navegación por pantallas o lectura de diapositivas.

Qué cuenta como avance real

Sí cuenta

Un flujo reducido pero ejecutable, con reglas, persistencia y pruebas coherentes.

Sí cuenta

Un prototipo técnico enfocado en un riesgo, con resultados reproducibles.

No basta

Mockups, diagramas o documentación sin software que se pueda ejecutar.

No basta

Pantallas aisladas, datos fijos o video sin instrucciones ni evidencia.

Cómo se revisará la entrega

Diseño y coherencia

¿Las vistas y ADRs corresponden al sistema que realmente existe?

Incremento funcional

¿El flujo funciona y representa reglas importantes del dominio?

Validación y evidencia

¿Puede reproducirse? ¿Hay pruebas y limitaciones explícitas?

Comunicación

¿El ZIP y el video permiten entender y defender el avance?

La nota se determinará con la rúbrica oficial de la evaluación; esta lámina resume el enfoque.

Checklist antes de enviar

Arquitectura actualizada

Vistas, ADRs, modelo de datos, interfaces y decisiones revisadas.

Software ejecutable

Código y dependencias; instrucciones de ejecución y datos sintéticos.

Pruebas y estado

Resultados, riesgos, pendientes, trazabilidad y alcance implementado.

Entrega completa

Un ZIP organizado + video de 5–6 minutos accesible para revisión.

Mensaje final: construir una parte, demostrarla bien

Una buena Evaluación 3 no promete un sistema completo: demuestra que una arquitectura justificada puede convertirse en software funcional.

La solicitud de cambio y la evolución posterior se evaluarán en la Evaluación 4.